ROS 2 MINI PROJECT

URDF Modeling of a 4 DOF Industrial Robotic Arm in ROS 2

Project report

Haresh Selvakumar
Department of Electronics and Communication Engineering
VIT Chennai
5 October 2026

Abstract

I developed a four-degree-of-freedom robotic arm model in ROS 2 Humble using URDF. The model contains six links, four revolute joints and one fixed tool attachment. Every link includes visual and collision geometry, and each movable joint has a defined axis and motion limits. The package builds successfully with colcon and displays the arm in RViz2. Joint sliders demonstrate different configurations, while a captured TF graph confirms a connected chain rooted at base_link.

Objective and scope

The project addresses all six questions in the mini project brief: robot structure, link geometry, joint definitions, assembly validation, RViz visualization and TF verification. The implementation models the geometry and kinematics of an illustrative industrial arm. It does not include motor control, physical hardware, motion planning or dynamic simulation.

Environment	Implementation
Operating system	Ubuntu 22.04.5 LTS in Oracle VirtualBox
ROS distribution	ROS 2 Humble
Model format	URDF XML
Build system	ament_cmake and colcon
Visualization	RViz2 and joint_state_publisher_gui
Package	four_dof_arm_description

Question 1 Robot Structure Design

The arm uses a serial kinematic chain. The base joint rotates the pedestal around its local Z axis. The shoulder, elbow and wrist joints rotate around their local Y axes. The tool is rigidly attached to the wrist. Four independently movable revolute joints provide four degrees of freedom; the fixed attachment adds none.

Order	Link	Connected joint	Function
0	base_link	Root	Stationary support
1	rotating_base_link	base_rotation_joint	Rotating pedestal
2	upper_arm_link	shoulder_joint	Upper arm segment
3	forearm_link	elbow_joint	Forearm segment
4	wrist_link	wrist_joint	Wrist segment
5	tool_link	tool_fixed_joint	Rigid tool block

Kinematic relationships

base_rotation_joint connects base_link to rotating_base_link. shoulder_joint connects rotating_base_link to upper_arm_link. elbow_joint connects upper_arm_link to forearm_link. wrist_joint connects forearm_link to wrist_link. tool_fixed_joint connects wrist_link to tool_link.

Reference configuration

At zero joint angles, all segments extend along positive Z. The visual model is 0.91 m tall: the 0.08 m base, 0.12 m pedestal, 0.30 m upper arm, 0.25 m forearm, 0.10 m wrist and 0.06 m tool form a continuous assembly. The tool_link frame is at 0.85 m above base_link; the tool geometry extends another 0.06 m above this frame.

These dimensions and limits are design choices for this educational model. The tool is represented by a block without movable gripper fingers. The link and joint names above are identical to those in the submitted URDF and captured TF tree.

Question 2 URDF Link Definition

Each link defines both a visual element and a collision element. Cylinders represent the base and pedestal; boxes represent the arm segments and tool. The collision geometry uses the same shape, dimensions and local origin as its corresponding visual geometry. All dimensions and translations are in metres.

Link	Geometry and dimensions	Local geometry origin
base_link	Cylinder radius 0.12; length 0.08	0 0 0.04
rotating_base_link	Cylinder radius 0.06; length 0.12	0 0 0.06
upper_arm_link	Box 0.06 x 0.06 x 0.30	0 0 0.15
forearm_link	Box 0.05 x 0.05 x 0.25	0 0 0.125
wrist_link	Box 0.04 x 0.04 x 0.10	0 0 0.05
tool_link	Box 0.08 x 0.04 x 0.06	0 0 0.03

Example from the submitted URDF

```
<link name="upper_arm_link">
  <visual>
    <origin xyz="0 0 0.15" rpy="0 0 0"/>
    <geometry><box size="0.06 0.06 0.30"/></geometry>
    <material name="orange"/>
  </visual>
  <collision>
    <origin xyz="0 0 0.15" rpy="0 0 0"/>
    <geometry><box size="0.06 0.06 0.30"/></geometry>
  </collision>
</link>
```

The link frame is located at the lower end of the upper arm. The origin shifts the centre of its 0.30 m box upward by 0.15 m, so the geometry extends from the joint to the next segment. rpy="0 0 0" leaves the geometry aligned with the link frame.

Named materials distinguish the parts: dark grey for the base, blue for the pedestal and forearm, orange for the upper arm and wrist, and silver for the tool. Inertial elements are omitted because this project uses kinematic visualization. Collision elements are present, but RViz alone does not perform collision checking.

Question 3 Joint Definition

All four movable joints use type="revolute". The origin specifies the joint location relative to its parent link, and the axis specifies the rotation direction in the joint frame. Every joint origin has zero roll, pitch and yaw.

Joint	Parent	Child	Origin xyz
base_rotation_joint	base_link	rotating_base_link	0 0 0.08
shoulder_joint	rotating_base_link	upper_arm_link	0 0 0.12
elbow_joint	upper_arm_link	forearm_link	0 0 0.30
wrist_joint	forearm_link	wrist_link	0 0 0.25
tool_fixed_joint	wrist_link	tool_link	0 0 0.10

Movable joint	Axis xyz	Lower rad	Upper rad	Effort Nm	Velocity rad per s
Base rotation	0 0 1	-3.14159	3.14159	30	1.0
Shoulder	0 1 0	-1.5708	1.5708	20	1.0
Elbow	0 1 0	-2.0944	2.0944	15	1.0
Wrist	0 1 0	-1.5708	1.5708	5	1.5

The angular ranges are approximately +/-180 degrees for the base, +/-90 degrees for the shoulder and wrist, and +/-120 degrees for the elbow. Effort and velocity values are model limits, not measured actuator ratings. Slider input demonstrates joint positions; it does not validate motor torque or velocity control.

Example revolute joint

```
<joint name="elbow_joint" type="revolute">
  <parent link="upper_arm_link"/>
  <child link="forearm_link"/>
  <origin xyz="0 0 0.30" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <limit lower="-2.0944" upper="2.0944"
         effort="15" velocity="1.0"/>
</joint>
```

The elbow pivot sits at the upper arm endpoint. The forearm rotates around local Y at this pivot. The fixed tool joint has no movable axis or motion limits, and preserves the tool pose relative to wrist_link.

Question 4 Assembly Validation

The complete model is stored in urdf/four_dof_arm.urdf under the robot name four_dof_industrial_arm. All joint references resolve to defined links. Each non-root link has one parent, and the assembled graph has a single root, base_link.

```
check_urdf urdf/four_dof_arm.urdf
```



Figure 1. check_urdf successfully parses the XML and prints the six-link hierarchy.

The parser reports Successfully Parsed XML and lists the chain through rotating_base_link, upper_arm_link, forearm_link, wrist_link and tool_link. This validates the URDF structure; geometric appearance is checked separately in RViz.

Package build

```
cd ~/ros2_arm_ws
colcon build --symlink-install --packages-select four_dof_arm_description
```

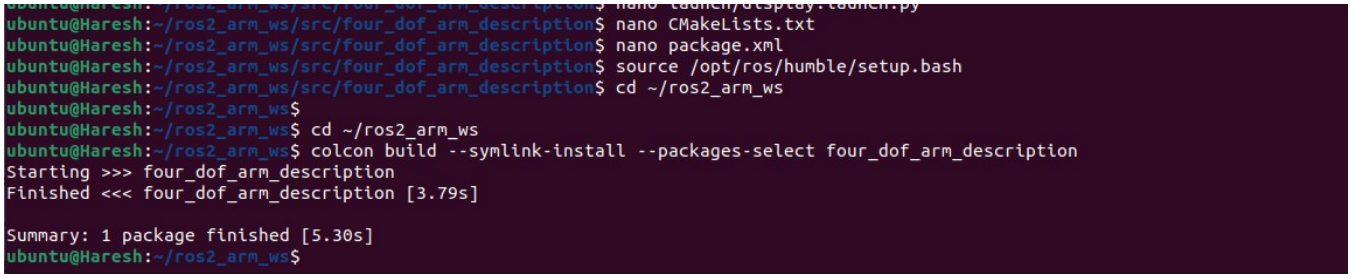


Figure 2. colcon completes the package build with one package finished.

CMake installs the urdf, launch and rviz directories into share/${PROJECT_NAME}. package.xml declares ament_cmake and the runtime dependencies required by the launch file. The successful build makes the model and visualization configuration available through the installed package path.

Question 5 RViz Visualization

The launch file starts robot_state_publisher, joint_state_publisher_gui and RViz2. It reads the URDF into the robot_description parameter and passes the saved rviz/arm.rviz configuration to RViz using the -d argument.

RViz setting	Configured value
Fixed Frame	base_link
RobotModel Description Source	Topic
RobotModel Description Topic	/robot_description
Visual geometry	Enabled
Collision geometry display	Disabled
TF display	Enabled with names and axes

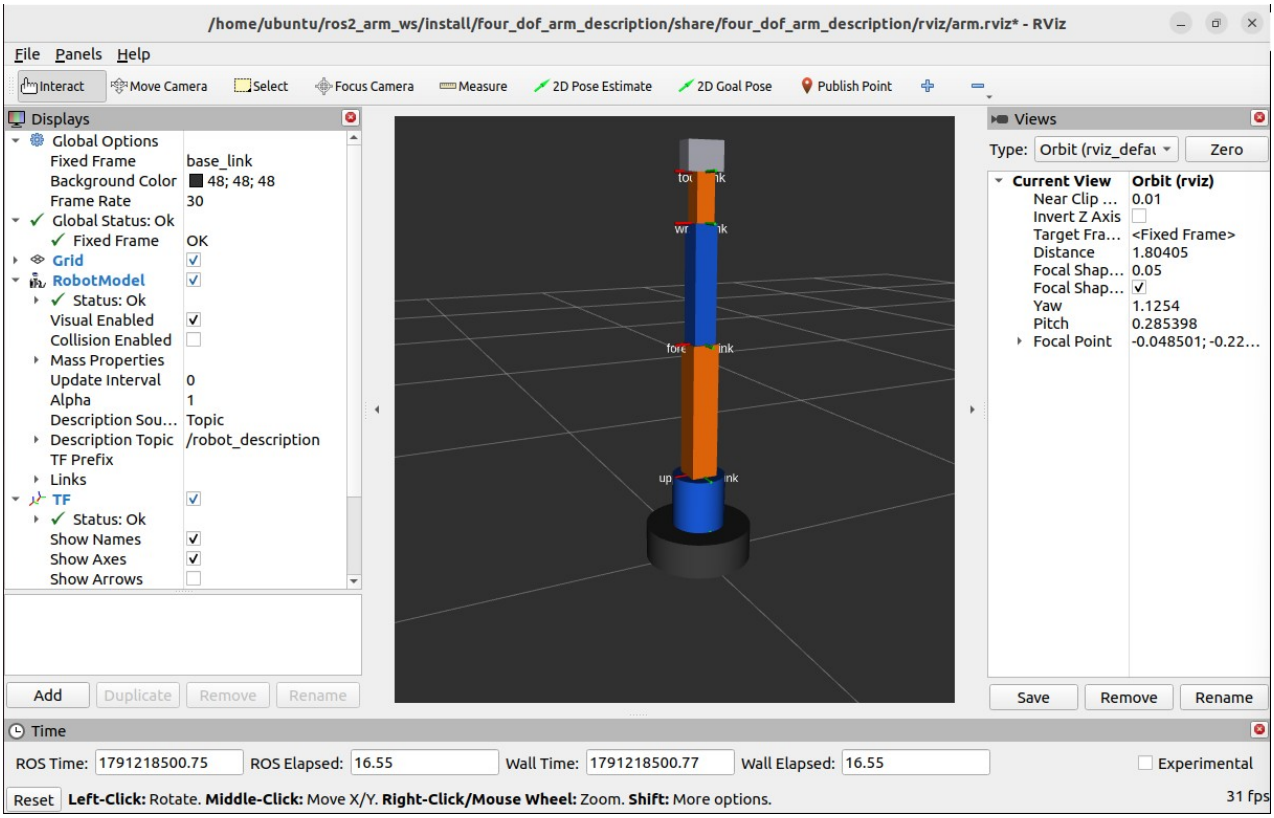


Figure 3. Relaunched RViz loads arm.rviz with the upright arm. Global Status, RobotModel and TF are all OK.

The reference pose displays the base, pedestal, upper arm, forearm, wrist and tool as an assembled model. The configuration loaded after a fresh launch, confirming that the fixed frame and display settings are reproducible. Joint positions restart at zero independently of the saved camera settings.

Question 5 Joint Position Demonstration

Changing the shoulder, elbow and wrist sliders produces bent configurations in RViz while maintaining the link connections. The screenshots below record the displayed joint values in radians.

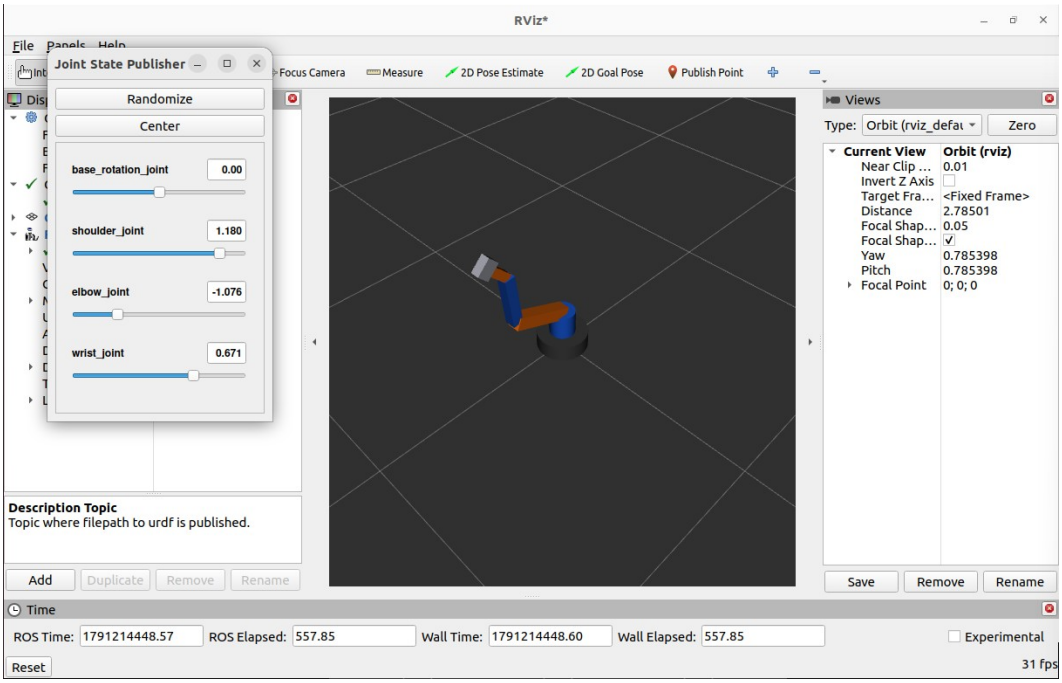


Figure 4. Bent pose A with base 0.000, shoulder 1.180, elbow -1.076 and wrist 0.671 rad.

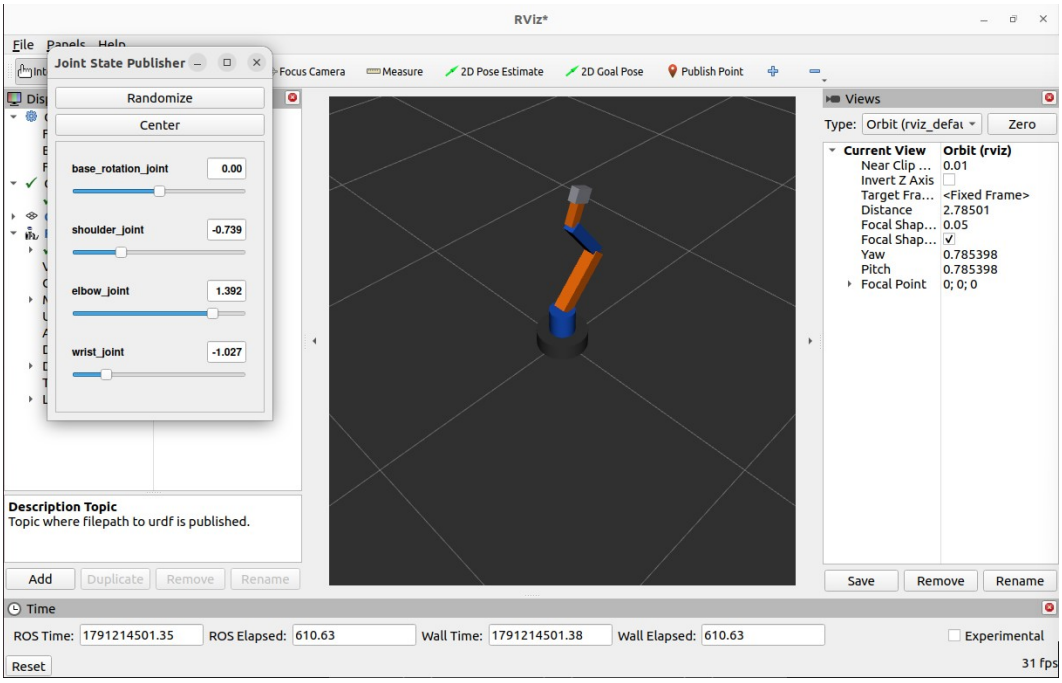


Figure 5. Bent pose B with base 0.000, shoulder -0.739, elbow 1.392 and wrist -1.027 rad.

The two poses demonstrate changes in the three pitch joints. The base rotation slider is present but remains at zero in these screenshots. All recorded angles are within the corresponding URDF bounds. These are kinematic demonstrations, without physics or collision avoidance.

Question 6 TF Tree Verification

With the launch running, ros2 run tf2_tools view_frames captures the live frame graph. The result is stored as evidence/tf_tree.pdf, with evidence/tf_tree.gv retaining the graph source. The captured tree matches the URDF chain and contains six frames connected by five transforms.

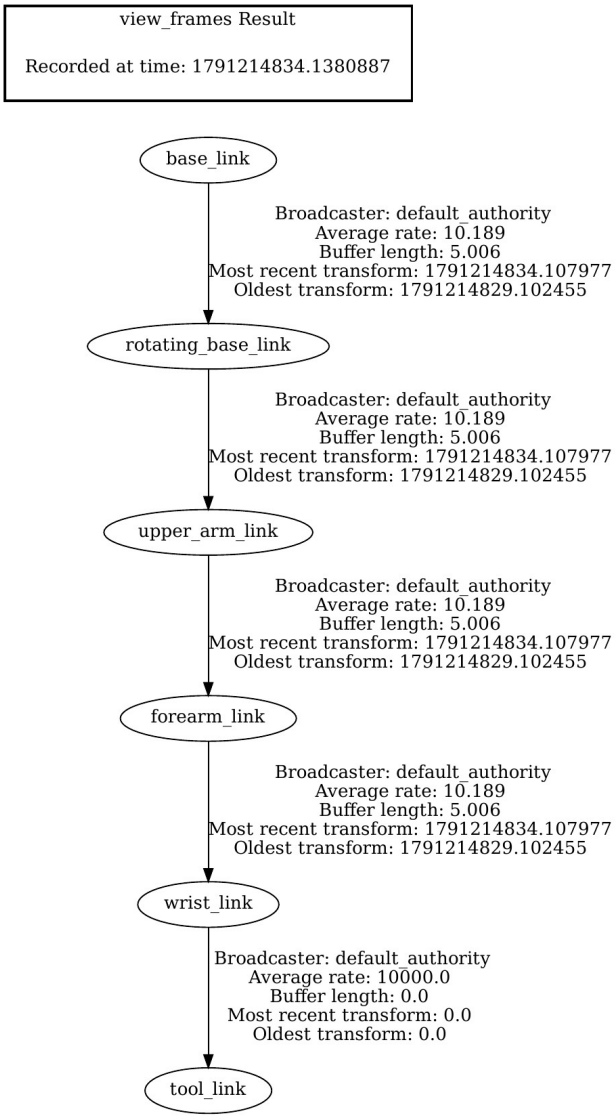


Figure 6. Captured ROS 2 TF tree with base_link as the root and tool_link as the terminal frame.

The four movable transforms show an average captured rate of about 10.189 Hz. The wrist_link to tool_link transform is fixed. Its displayed rate of 10000.0 and zero timestamps are static-transform reporting values, not a 10 kHz actuator or sensor rate.

Question 6 Frame Display and Interpretation

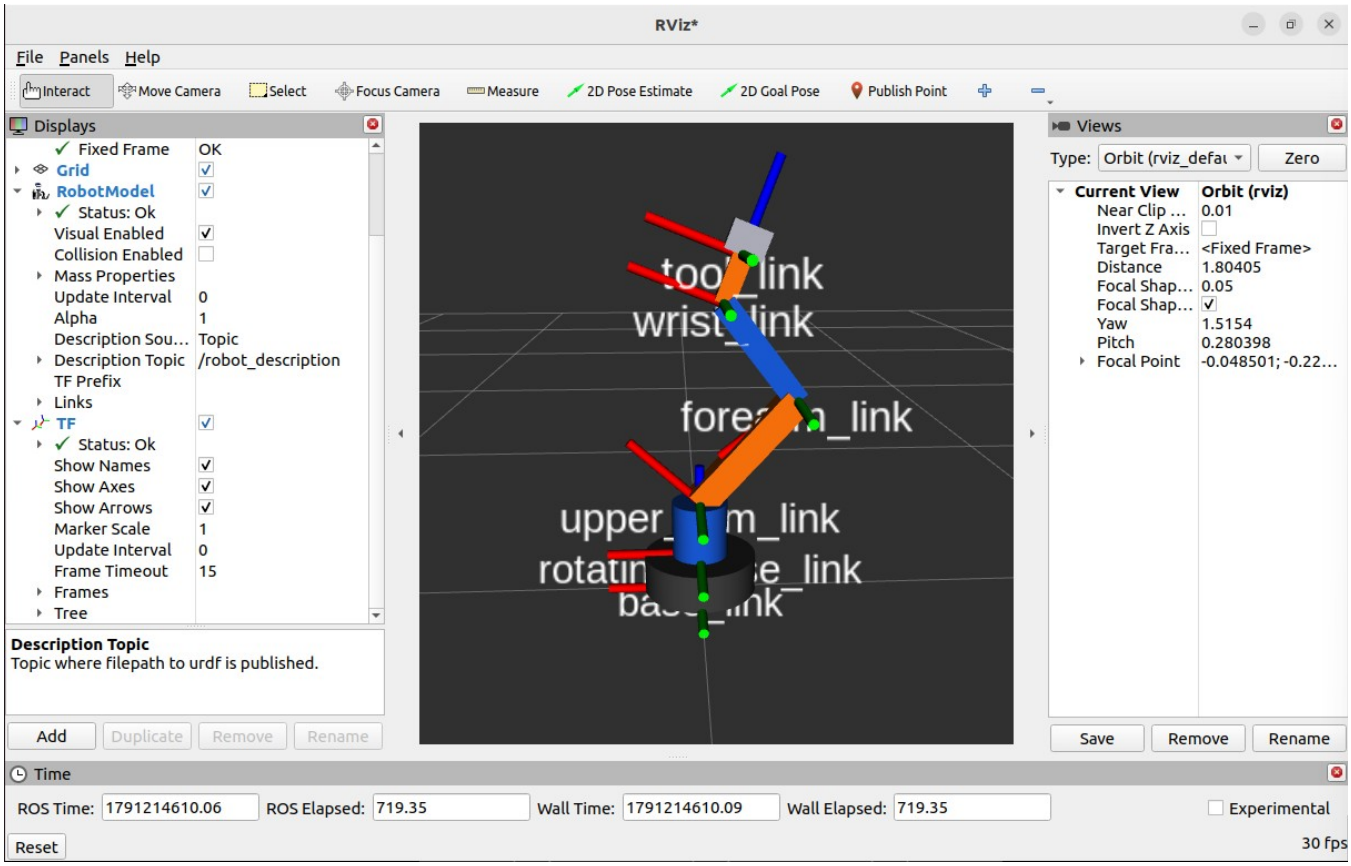


Figure 7. RViz displays link frame names and axes, with RobotModel and TF status OK.

Every frame is reachable from base_link. Each child has one parent and no cycle or disconnected branch appears in the captured graph. This verifies the structure needed to transform poses between the base, arm segments and tool.

robot_state_publisher combines the model with joint state updates to calculate link transforms. The movable joints are represented on /tf and the fixed tool attachment on /tf_static [2]. The slider window supplies joint positions, and RViz uses the transforms to place the model geometry.

Meaning of the axes

The frame axes indicate local orientation: red represents X, green Y and blue Z. Their location follows the joint frame origins rather than the geometry centres. Consequently, a link can have a frame at its lower endpoint and geometry shifted upward by half its length.

Build and Execution Procedure

The source package is organized in a ROS 2 workspace. For reproduction, copy the submitted package into a workspace src directory. On an Ubuntu system with ROS 2 Humble installed, install any missing dependencies before building.

```
sudo apt install ros-humble-rviz2 ros-humble-robot-state-publisher \
  ros-humble-joint-state-publisher-gui ros-humble-tf2-tools \
  python3-colcon-common-extensions liburdfdom-tools graphviz
```

Build and launch

```
source /opt/ros/humble/setup.bash
cd ~/ros2_arm_ws
colcon build --symlink-install --packages-select four_dof_arm_description
source install/setup.bash
ros2 launch four_dof_arm_description display.launch.py
```

The first source command loads ROS 2. colcon builds and installs the package. The second source command adds this workspace to the environment, and ros2 launch starts the three visualization nodes. Keep the launch terminal running while inspecting the arm.

Generate the TF evidence

```
# In a second terminal while the launch is running
source /opt/ros/humble/setup.bash
source ~/ros2_arm_ws/install/setup.bash
mkdir -p ~/ros2_arm_ws/evidence
cd ~/ros2_arm_ws/evidence
ros2 run tf2_tools view_frames
```

The tool listens for approximately five seconds. In the tested installation, generated filenames include a timestamp, for example frames_2026-10-05_21.10.34.pdf. The submitted copy is named tf_tree.pdf.

File organization

Path within source package	Purpose
urdf/four_dof_arm.urdf	Complete link and joint model
launch/display.launch.py	Starts publishers and RViz with saved settings
rviz/arm.rviz	Fixed frame, display and camera configuration
CMakeLists.txt	ament build and installation rules
package.xml	Package metadata and dependencies

Results and Conclusion

Brief requirement	Result and evidence
Correct link and joint definitions	Six links and five joints; connected serial chain
Visual and collision elements	Both present for every link in the URDF
Proper types and limits	Four bounded revolute joints and one fixed joint
Complete assembled URDF	Successful check_urdf parsing in Figure 1
Successful build	One package finished in Figure 2
RViz visualization	Upright and bent poses in Figures 3 to 5
Correct TF tree	Six connected frames in Figures 6 and 7

Conclusion

The implementation meets the modeling and visualization requirements of the mini project. The URDF describes an assembled four-DOF arm, the package builds, and the saved RViz configuration displays the model and its frame axes. The captured TF tree verifies the expected parent-child relationships from base_link to tool_link.

Limitations and possible extensions

The model uses simple primitive geometry and illustrative limits. It contains no masses or inertia tensors, hardware interfaces, controllers or motion planner. Collision geometry is defined, but collision-free motion and physical performance have not been evaluated. Future work could add inertial properties, a simulated controller and motion planning after the present kinematic model is retained as a baseline.

References

[1] Supplied mini project brief, ROS 2 URDF of 4-DOF Industrial Arm, pages 1 and 2. The six question sections in this report follow its task list.

[2] ROS robot_state_publisher repository and documentation. https://github.com/ros/robot_state_publisher (accessed 5 October 2026).

[3] ROS joint_state_publisher repository. https://github.com/ros/joint_state_publisher (accessed 5 October 2026).

[4] Project implementation and evidence: four_dof_arm.urdf, display.launch.py, arm.rviz, package.xml, CMakeLists.txt, tf_tree.pdf and the recorded Ubuntu and RViz screenshots.